

Datalink Access — Concise Lecture Notes + Sample Programs

Based on the uploaded *DL Case Study.pdf*, Chapter 29. The sample programs below are compact teaching versions/excerpts of the mechanisms illustrated in the source; they are intended to make the lecture code-readable rather than reproduce every support routine.

1. Datalink-Layer Access

- Applications can watch packets received by the datalink layer; this enables programs such as tcpdump to run on ordinary systems.
- Promiscuous mode can make an interface receive packets beyond those addressed to the host. In switched networks, monitoring other ports generally requires monitor mode / port mirroring.
- Applications can also read/write non-IP datalink traffic; the source gives a Unix RARP server as an example.

2. OS-Specific Access

- **BPF**: BSD Packet Filter — capture, filtering, buffering and timestamps.
- **DLPI**: SVR4 Datalink Provider Interface — protocol-independent datalink access through STREAMS.
- **SOCK_PACKET**: original Linux datalink packet-receive interface.
- **PF_PACKET**: newer Linux packet-socket family with additional filtering/performance features.

3. BPF — What to Remember

- BPF is called immediately after packet reception and as late as possible before transmission to obtain accurate timestamps.
- Each application can load its own filter. BPF uses a register-based filter machine.
- Kernel filtering, snapshot length (snaplen), buffering and double buffering reduce data-copy and system-call overhead.

4. DLPI and Linux

- DLPI uses pfmod for kernel filtering and bufmod for buffering. The source contrasts BPF's directed acyclic control-flow graph with pfmod's Boolean expression tree.
- PF_PACKET supports SOCK_DGRAM (cooked packet) and SOCK_RAW (complete link-layer packet).
- ETH_P_ALL selects all protocols; ETH_P_IP selects IPv4; ETH_P_ARP and ETH_P_IPV6 are other examples.
- The source notes limitations in Linux buffering, kernel filtering and device-specific filtering.

5. libpcap and libnet

- **libpcap** gives implementation-independent access to OS packet-capture facilities and is used by tcpdump.
- **libnet** provides an implementation-independent interface for crafting/injecting packets and hides many header-construction details.

6. UDP Checksum Case Study

- Goal: build a UDP datagram containing a DNS query, send it with a raw socket, capture the reply with libpcap, and inspect the UDP checksum field.
- With IPv4, the source describes UDP checksum computation as optional; a received checksum of zero is interpreted by the program as 'checksums off'.
- The program therefore combines two different mechanisms: raw packet generation and packet capture.

Sample Program 1 — BPF Filter Expression

The source demonstrates that BPF filters can be simple or can inspect protocol-header fields. The following is the filter syntax shown in the chapter:

```
/* Select TCP traffic to/from port 80
with SYN, FIN or RST set */
tcp and port 80 and tcp[13:1] & 0x7 != 0
```

Teaching point: `tcp[13:1]` refers to the one-byte value at byte offset 13 of the TCP header. The filter is compiled by `pcap_compile()` in the later `libpcap` example.

Sample Program 2 — PF_PACKET: Receive IPv4 Frames

```
#include <sys/socket.h>
#include <linux/if_packet.h>
#include <net/ethernet.h>
#include <arpa/inet.h>

int main(void)
{
    int fd;

    fd = socket(PF_PACKET, SOCK_RAW,
        htons(ETH_P_IP));

    if (fd < 0)
        return 1;

    /* receive frames with recvfrom(fd, ...) */

    return 0;
}
```

The source gives this socket form for receiving IPv4 frames. A privileged process is required. With `SOCK_RAW`, the complete link-layer packet is available.

Sample Program 3 — Minimal libpcap Capture Setup

```
pcap_t *pd;
struct bpf_program fcode;
char errbuf[PCAP_ERRBUF_SIZE];

/* Open a live capture device */
pd = pcap_open_live(device, snaplen, 0, timeout, errbuf);

/* Compile a filter */
pcap_compile(pd, &fcode, filter_string, 0, netmask);

/* Install the filter */
pcap_setfilter(pd, &fcode);

/* Identify the datalink type */
datalink = pcap_datalink(pd);
```

This condenses the sequence described around `open_pcap()`: open the device, obtain network information, compile/install a filter, and determine the datalink type.

Sample Program 4 — Raw Socket + IP_HDRINCL

```
int rawfd;
int on = 1;

/* Create raw socket */
rawfd = socket(dest->sa_family, SOCK_RAW, 0);

/* Allow application to supply IP header */
setsockopt(rawfd, IPPROTO_IP, IP_HDRINCL,
&on, sizeof(on));
```

The source notes that the process needs superuser privileges for this raw-socket operation. IP_HDRINCL allows the program to write complete IP datagrams, including the IP header.

Sample Program 5 — Building the DNS Query

```
char *ptr;
char *buf;
size_t nbytes;

buf = Malloc(sizeof(struct udphdr) + 100);
ptr = buf + sizeof(struct udphdr);

/* DNS identification */
*((uint16_t *)ptr) = htons(1234);
ptr += 2;

/* Flags: recursion desired */
*((uint16_t *)ptr) = htons(0x0100);
ptr += 2;

/* One question; no answers/authority/additional */
*((uint16_t *)ptr) = htons(1);
ptr += 2;
*((uint16_t *)ptr) = 0; ptr += 2;
*((uint16_t *)ptr) = 0; ptr += 2;
*((uint16_t *)ptr) = 0; ptr += 2;

/* a.root-servers.net + A query + class 1 */
memcpy(ptr, "\001a\014root-servers\003net\000", 20);
ptr += 20;
*((uint16_t *)ptr) = htons(1); ptr += 2;
*((uint16_t *)ptr) = htons(1); ptr += 2;

nbytes = (ptr - buf) - sizeof(struct udphdr);
udp_write(buf, nbytes);
```

The source's function creates 36 bytes of DNS user data: the DNS header fields plus the encoded domain name, query type and query class. It then calls `udp_write()` to construct the UDP/IP headers.

Sample Program 6 — Reading the Captured Packet

```
char *ptr;
int len;

for (;;) {
ptr = next_pcap(&len);

switch (datalink) {
case DLT_NULL:
return udp_check(ptr + 4, len - 4);

case DLT_EN10MB:
return udp_check(ptr + 14, len - 14);

case DLT_SLIP:
case DLT_PPP:
return udp_check(ptr + 24, len - 24);

default:
err_quit("unsupported datalink");
}
}
```

The offsets shown are those used in the source example: 4 bytes for loopback, 14 for Ethernet, and 24 for the cited SLIP/PPP examples. The program first determines the datalink type with `pcap_datalink()`.

Sample Program 7 — Verifying the IP/UDP Packet

```
struct ip *ip;
int hlen;

if (len < sizeof(struct ip) + sizeof(struct udphdr))
err_quit("packet too short");

ip = (struct ip *)ptr;

if (ip->ip_v != IPVERSION)
err_quit("wrong IP version");

hlen = ip->ip_hl << 2;

if (hlen < sizeof(struct ip))
err_quit("invalid IP header length");

if (len < hlen + sizeof(struct udphdr))
err_quit("packet too short");

if (in_cksum((uint16_t *)ip, hlen) != 0)
err_quit("IP checksum error");

if (ip->ip_p == IPPROTO_UDP)
return (struct udpiphdr *)ip;

err_quit("not a UDP packet");
```

This follows the verification logic of `udp_check()`: length, IP version, IP-header length, IP checksum, and UDP protocol field are checked before the UDP/IP structure is returned.

Sample Program 8 — Timeout and Checksum Decision

```
canjump = 1;
send_dns_query();
nsent++;

alarm(timeout);
ui = udp_read();

canjump = 0;
alarm(0);

if (ui->ui_sum == 0)
printf("UDP checksums off\n");
else
printf("UDP checksums on\n");

if (verbose)
printf("received UDP checksum = %x\n",
ntohs(ui->ui_sum));
```

The source uses `SIGALRM` with `sigsetjmp/siglongjmp` for timeout handling. The timeout sequence is 3 s, 6 s, then 12 s. If the captured UDP checksum is zero, the program prints "UDP checksums off"; otherwise it reports that checksums are on.

Sample Program 9 — libnet Packet Construction

```
libnet_t *l;
char errbuf[LIBNET_ERRBUF_SIZE];

/* Initialize libnet for IPv4 raw output */
l = libnet_init(LIBNET_RAW4, NULL, errbuf);

dns_tag = libnet_build_dns_v4(
1234, 0x0100, 1, 0, 0, 0,
qbuf, 24, l, dns_tag);

udp_tag = libnet_build_udp(
local_port, dest_port,
packet_size, 0,
NULL, 0, l, udp_tag);

/* 0 checksum argument => libnet calculates it */

ip_tag = libnet_build_ipv4(
packet_size + LIBNET_IPV4_H,
0, 0, 0, TTL_OUT, IPPROTO_UDP,
0, local_addr, dest_addr,
NULL, 0, l, ip_tag);

libnet_write(l);
```

The source explains that `libnet` accepts header fields as function arguments and handles packet assembly. A checksum argument of zero causes automatic checksum calculation; `libnet_toggle_checksum()` can disable it.